

M5 — Aprendizaje científico: Neural ODE y PINN

Redes que aprenden física: dinámica vs. solución, y por qué funcionan

Proyecto Wilson–Cowan — Iniciación a la I+D ITBA

12 de julio de 2026

Índice

1. El problema y las dos rutas	1
2. Redes neuronales: MLP, activaciones y sesgo espectral	2
2.1. El MLP como aproximador de funciones	2
2.2. Sesgo espectral y Fourier features	2
3. Neural ODE: aprender la dinámica	3
3.1. Integración y su costo (recordatorio de M3)	3
3.2. Backprop a través del solver	3
3.3. Enfoque gray-box / UDE	4
3.4. Multiple shooting	4
4. PINN: aprender la solución	4
5. Reparametrización softplus para positividad	5
6. PINN vs. Neural ODE: cuándo cada una	5
7. Conexión con la identificabilidad y el control	7

Resumen

Este documento (núcleo del paquete) explica las dos arquitecturas de *Scientific Machine Learning* (SciML) que usamos para identificar los 10 parámetros de Wilson–Cowan (WC): la **Neural ODE**, que aprende la *dinámica* $\dot{\mathbf{x}} = f_{\theta}(\mathbf{x})$ e integra para obtener la trayectoria, y la **PINN**, que aprende directamente la *solución* $t \mapsto \mathbf{x}(t)$ penalizando el residuo físico de las ecuaciones. Antes vemos los ingredientes: qué es un MLP y por qué sufre *sesgo espectral*, cómo lo curamos con *Fourier features*, cómo se propaga el gradiente a través del integrador (*adjoint* vs. backprop directo), el enfoque *gray-box*/UDE, el *multiple shooting* y la reparametrización *softplus* para positividad. Cerramos con una tabla comparativa y la conexión con el control: la Neural ODE sirve de gemelo digital de la planta, la PINN no.

Requiere: M2 (EDOs y sistemas dinámicos), M3 (integración numérica y rigidez), M4 (optimización y descenso por gradiente). **Conecta con:** B5 (control en lazo cerrado) y P4 (código `neural_ode`).

1. El problema y las dos rutas

El estado de WC es $\mathbf{x} = [I, E]$ (tasas de disparo inhibitoria y excitatoria) y su dinámica, con offsets de reposo k_e, k_i y sigmoidea $S_a(u) = 1/(1 + e^{-au})$ (ver M2, B3):

$$\dot{I} = \frac{1}{\tau_i} (-I + S_i(w_{IE}E - w_{II}I + Q(t) - \theta_i) - k_i), \quad (1)$$

$$\dot{E} = \frac{1}{\tau_e} (-E + S_e(w_{EE}E - w_{EI}I + P(t) - \theta_e) - k_e), \quad (2)$$

con $P(t) \rightarrow E$, $Q(t) \rightarrow I$ los estímulos y θ los 10 parámetros a recuperar. La salida observada es $y(t) = E(t) - I(t)$. **Identificar** es hallar θ a partir de trayectorias medidas con ruido.

Hay dos maneras de meter la física en una red:

- **Neural ODE:** aprender la *regla de cambio* $f_\theta : (\mathbf{x}, P, Q) \mapsto \dot{\mathbf{x}}$ e *integrarla* para reproducir la trayectoria. El estímulo es entrada explícita.
- **PINN:** aprender la *trayectoria* $\mathbf{x}_\theta : t \mapsto \mathbf{x}(t)$ como función del tiempo, exigiendo que satisfaga (1)–(2) vía un término de pérdida. El estímulo queda implícito (una red por trayectoria).

Ambas comparten el mismo bloque constructor — un perceptrón multicapa — y la misma herramienta que las hace posibles: la *diferenciación automática*. Empezamos por ahí.

2. Redes neuronales: MLP, activaciones y sesgo espectral

2.1. El MLP como aproximador de funciones

Un *perceptrón multicapa* (MLP) es una función parametrizada que compone capas afines seguidas de una no linealidad σ [6]. Con entrada $\mathbf{z}_0$ y capas $\ell = 1, \dots, L$:

$$\mathbf{z}_\ell = \sigma(W_\ell \mathbf{z}_{\ell-1} + \mathbf{b}_\ell), \quad \text{MLP}(\mathbf{z}_0) = W_L \mathbf{z}_{L-1} + \mathbf{b}_L. \quad (3)$$

Los pesos W_ℓ y sesgos $\mathbf{b}_\ell$ son los parámetros aprendibles. El *teorema de aproximación universal* garantiza que, con suficientes neuronas, un MLP puede aproximar cualquier función continua en un compacto tan bien como se quiera; en la práctica lo que importa no es la existencia sino qué tan *fácil* es que el descenso por gradiente (ver M4) encuentre esa aproximación.

Funciones de activación. La no linealidad σ es lo que distingue un MLP de una regresión lineal (sin ella, la composición de capas afines colapsa a una sola afín). Las usuales:

- **tanh / sigmoidea:** suaves y acotadas, con derivadas de todo orden. *Convenientes para PINN*, porque el residuo físico (9) deriva la red respecto de t por autograd y una activación suave da derivadas limpias.
- **ReLU** ($\max(0, x)$): barata y sin saturación, domina en visión; pero su segunda derivada es nula casi en todos lados, lo que la vuelve pobre cuando se necesitan derivadas de la salida.
- **softplus** ($\log(1 + e^x)$): versión suave de ReLU; la reusaremos en la Sección 5 para forzar positividad de parámetros.

2.2. Sesgo espectral y Fourier features

Un MLP entrenado con descenso por gradiente exhibe un *sesgo espectral* (*spectral bias*): aprende primero las componentes de **baja frecuencia** de la función objetivo y le cuesta muchísimo capturar las de alta frecuencia [5]. Intuitivamente, los modos suaves reducen la pérdida más rápido, así que el optimizador los prioriza y las oscilaciones rápidas quedan sin representar.

Esto es un problema directo para nosotros: en el régimen *theta-gamma* objetivo, la salida $y(t)$ tiene ráfagas gamma rápidas moduladas por una envolvente theta lenta. Una PINN que mapea $t \mapsto \mathbf{x}$ con un MLP crudo aplanaría el gamma.

La cura: Fourier features. En vez de alimentar la red con t directamente, se mapea t a un vector de senos y cosenos con un banco de frecuencias $B = \{b_1, \dots, b_m\}$ *antes* de la primera capa:

$$\gamma(t) = [\sin(2\pi b_1 t), \cos(2\pi b_1 t), \dots, \sin(2\pi b_m t), \cos(2\pi b_m t)]. \quad (4)$$

Con esta codificación, representar una oscilación de frecuencia b_k ya no exige que el MLP la “invente”: se la damos servida como coordenada de entrada. Así la red puede aprender los ritmos

rápidos del modelo sin necesidad de crecer en tamaño, y el sesgo espectral deja de ser una barrera. La Figura 1 lo ilustra.

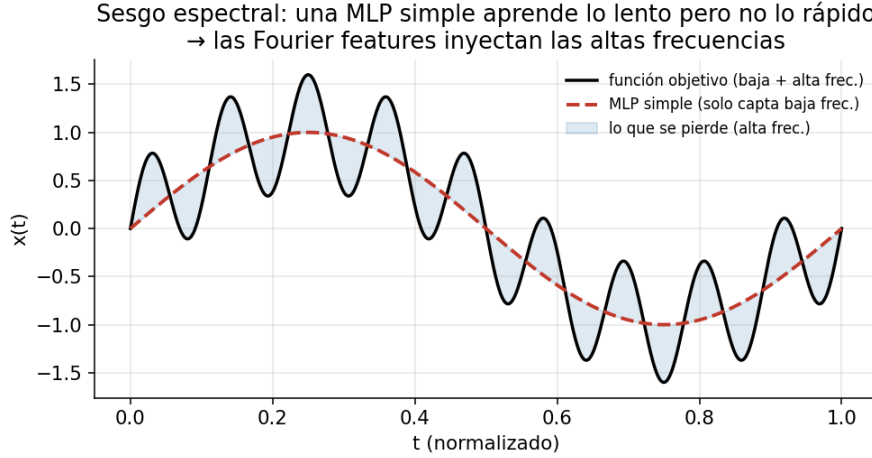


Figura 1: Sesgo espectral: un MLP simple capta la componente de baja frecuencia de la señal pero pierde la de alta frecuencia; el mapeo por *Fourier features* le entrega las oscilaciones rápidas como coordenadas de entrada y permite recuperarlas.

3. Neural ODE: aprender la dinámica

Definición 1 (Neural ODE). *Modelo en el que la derivada del estado da una función parametrizada, y la salida se obtiene **integrando** esa derivada [1]:*

$$\dot{\mathbf{x}} = f_{\theta}(\mathbf{x}, t), \quad \mathbf{x}(t_1) = \mathbf{x}(t_0) + \int_{t_0}^{t_1} f_{\theta}(\mathbf{x}(t), t) dt. \quad (5)$$

La intuición nace de las ResNets: un bloque residual $\mathbf{h}_{n+1} = \mathbf{h}_n + f(\mathbf{h}_n)$ es exactamente *un paso de Euler* de una EDO; al hacer los pasos infinitesimales, la red profunda tiende a un límite continuo, y ese límite es la Neural ODE [1]. En vez de aprender la función entrada→salida, aprende su *tasa de cambio*.

3.1. Integración y su costo (recordatorio de M3)

Para evaluar la salida hay que *resolver* la EDO numéricamente. El proyecto usa **RK4 de paso fijo** (orden 4, 4 evaluaciones de f por paso, diferenciable). El costo se mide en *número de evaluaciones de f* (NFE): con RK4, $\text{NFE} = 4 \times \text{pasos}$. Cuán chico debe ser Δt lo fija la *rigidez* del sistema (ver M3); WC es poco rígido y admite pasos grandes, pero con estímulos conmutados (pulsos, escalones optogenéticos) un Δt grande degrada la identificación por el error del retenedor de orden cero en cada conmutación.

3.2. Backprop a través del solver

Entrenar significa integrar f_{θ} y minimizar el error contra la trayectoria medida (*rollout*):

$$\mathcal{L}(\theta) = \|\text{rollout}(f_{\theta}, \mathbf{x}_0, P, Q, \Delta t) - \mathbf{y}_{\text{medido}}\|^2. \quad (6)$$

Como el integrador RK4 es una *composición de operaciones diferenciables*, se puede propagar el gradiente $\partial \mathcal{L} / \partial \theta$ por *backpropagation a través del solver* (autograd desenrolla todos los pasos). Es simple y exacto, pero guarda en memoria todas las etapas intermedias del rollout.

El método adjoint. La alternativa que popularizó el paper original [1] evita guardar las activaciones: calcula el gradiente resolviendo *una segunda EDO hacia atrás* en el tiempo (la ecuación *adjunta*)

$$\dot{\mathbf{a}}(t) = -\mathbf{a}(t)^\top \frac{\partial f_\theta}{\partial \mathbf{x}}, \quad (7)$$

de modo que todos los gradientes salen de una sola pasada del solver sobre un estado aumentado, con memoria $O(1)$ respecto de la longitud de la trayectoria. Es la técnica de elección para horizontes largos. **En este proyecto usamos backprop directo:** como entrenamos por trayectorias *cortas* (ver *multiple shooting* abajo), la memoria no es el cuello de botella y el desenrollado directo es más simple y estable.

3.3. Enfoque gray-box / UDE

En lugar de una red genérica de caja negra ($f_\theta = \text{MLP}$ puro), usamos un *backbone* con la **estructura exacta** de (1)–(2) y parámetros θ aprendibles, más una corrección neuronal *opcional* g_ϕ — es una *Universal Differential Equation* (UDE) [3]:

$$f_\theta(\mathbf{x}, P, Q) = \underbrace{f_{\text{WC}}(\mathbf{x}, P, Q; \theta)}_{\text{física conocida}} + \underbrace{g_\phi(\mathbf{x}, P, Q)}_{\text{corrección (off por defecto)}}. \quad (8)$$

Ventajas de este *gray-box*:

- (I) **Parámetros interpretables:** lo que se aprende son los pesos y umbrales de WC, no pesos opacos de una red — se pueden leer, comparar con la biología y diagnosticar con Fisher+SVD (ver M6).
- (II) **Estructura para el control:** conservar la forma de las ecuaciones permite que el controlador cancele el acoplamiento E–I y prediga la respuesta a P, Q (ver B5).
- (III) **Costo despreciable de red:** con la corrección *apagada* (el defecto), el “modelo” son ~ 10 escalares. Todo el costo computacional está en la *integración*, no en pasar datos por una red; entrenar es casi tan barato como simular WC.

La corrección g_ϕ se activa solo cuando se sospecha que los datos no son exactamente WC (dinámica no modelada): entonces g_ϕ absorbe el residuo estructural sin romper la interpretabilidad del backbone.

3.4. Multiple shooting

Integrar *de un tirón* una trayectoria larga es numéricamente inestable: pequeños errores de parámetro se amplifican a lo largo del rollout y el gradiente “explota” (la sensibilidad de la solución final respecto de θ crece de forma casi exponencial en dinámicas oscilatorias). La cura clásica [4] es el *multiple shooting*: se parte la trayectoria en **ventanas cortas**, cada una **reiniciada desde el estado observado** en su borde inicial, y solo se integra dentro de la ventana.

- Cada ventana es un horizonte corto $\Rightarrow$ el gradiente no explota y el problema queda mejor condicionado.
- Reduce la memoria del backprop (y justifica usar backprop directo en vez de adjoint).
- Precio: un pequeño sesgo por los reinicios, ya que el estado observado tiene ruido. En la práctica es un intercambio muy favorable: estabilidad de entrenamiento a cambio de una perturbación acotada.

4. PINN: aprender la solución

Definición 2 (PINN — *Physics-Informed Neural Network*). Una red que aproxima la solución $\mathbf{x}_\theta(t) \approx [I(t), E(t)]$ como función del tiempo, entrenada con dos términos [2]:

$$\mathcal{L} = \underbrace{\lambda_d \|\mathbf{x}_\theta(t_i) - \mathbf{y}_i\|^2}_{\text{datos}} + \underbrace{\lambda_p \|\dot{\mathbf{x}}_\theta(t) - f_{\text{WC}}(\mathbf{x}_\theta(t); \theta)\|^2}_{\text{residuo físico}}. \quad (9)$$

El primer término ajusta la red a los datos observados; el segundo la obliga a *satisfacer las ecuaciones de WC* en un conjunto de puntos de colocación. Los parámetros θ se optimizan **junto** con los pesos de la red: la física no es solo un chequeo, es lo que hace identificable a θ .

La derivada por autograd (clave para el ruido). El $\dot{\mathbf{x}}_\theta(t)$ del residuo no se calcula por diferencias finitas sobre los datos, sino por *diferenciación automática* de la red respecto de su entrada t . Esto es central: estimar $\dot{\mathbf{x}}$ por diferencias finitas sobre señales ruidosas *amplifica* el ruido (dividir diferencias chicas), mientras que autograd deriva la función suave $\mathbf{x}_\theta$, que la red ya suavizó al ajustar los datos. Es la ventaja frente a los métodos de *derivative matching* clásicos (tipo SINDy), que sí necesitan derivar los datos y sirven de baseline de comparación.

Fourier features, otra vez. Como la PINN mapea $t \mapsto \mathbf{x}$ con un MLP, sufre el sesgo espectral de la Sección 2. Por eso también aquí se antepone *Fourier features* $\gamma(t)$ a la red, para captar los ritmos rápidos del régimen theta-gamma.

Lecciones de entrenamiento (medidas en el proyecto). El balance λ_d, λ_p es delicado: con el peso físico demasiado alto la red cae en un *mínimo degenerado* (satisface la física con parámetros errados); con el de datos demasiado alto los parámetros no se mueven. Entrenar los dos términos en *etapas separadas* rompe el acoplamiento (gradiente ≈ 0). La receta que funciona es **entrenamiento conjunto** (datos + física a la vez), **multi-trayectoria** (un mismo θ compartido por varias trayectorias) y λ_d alto, con Fourier features.

5. Reparametrización softplus para positividad

Varios parámetros de WC deben ser *estrictamente positivos*: las constantes de tiempo τ_e, τ_i y las ganancias sigmoideas a_e, a_i . En vez de imponer una restricción dura al optimizador (que introduce fronteras donde el gradiente se comporta mal), se aprende una variable *libre* $\rho \in \mathbb{R}$ y se define el parámetro vía *softplus*:

$$\theta = \text{softplus}(\rho) = \log(1 + e^\rho) > 0. \quad (10)$$

Es suave, monótona e invertible, así que el gradiente fluye sin barreras y el parámetro nunca cruza el cero. El optimizador de M4 trabaja en ρ (sin restricciones) y la física ve siempre un θ válido.

Offsets k_e, k_i derivados, no aprendidos. Los offsets de reposo de (1)–(2) **no se aprenden libres**. Se *derivan* de los otros parámetros para que el reposo $E = I = 0$ sea equilibrio sin estímulo:

$$k_e = \frac{1}{1 + e^{a_e \theta_e}}, \quad k_i = \frac{1}{1 + e^{a_i \theta_i}}. \quad (11)$$

Al restarlos, con $P = Q = 0$ el término no lineal cancela exactamente el estado nulo y $\dot{E} = \dot{I} = 0$. Dejarlos libres agregaría una *degeneración* extra (dos parámetros que compensan lo mismo), justo lo que queremos evitar en un problema de identificación ya mal condicionado (ver M6).

6. PINN vs. Neural ODE: cuándo cada una

Las dos arquitecturas resuelven el mismo problema por caminos opuestos. La Figura 2 resume la comparación del proyecto; la Tabla 1 la sistematiza.

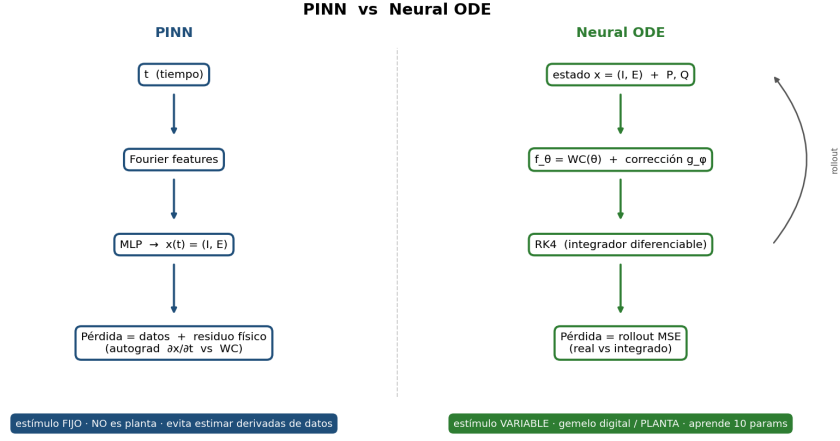


Figura 2: Comparación PINN vs. Neural ODE en la identificación de Wilson–Cowan (figura del proyecto): qué aprende cada una y su desempeño relativo.

	PINN	Neural ODE
Qué aprende	solución $t \mapsto \mathbf{x}$	dinámica $(\mathbf{x}, P, Q) \mapsto \dot{\mathbf{x}}$
Derivadas	autograd en t	integración numérica (rollout)
Estímulo	fijo (una red por trayectoria)	variable / en línea (entrada explícita)
Uso en control	no directo	sí (gemelo digital / planta)
Costo dominante	tamaño de red	integración (NFE)
Robustez a ruido	suaviza nativamente	vía suavizado + ventanas (shooting)
Denoising	implícito en la red	requiere multiple shooting

Cuadro 1: Comparación operativa PINN vs. Neural ODE.

Resultados medidos. En corridas limpias ambas identifican los 4 pesos con error sub-porcentual; en el proyecto la Neural ODE llegó a identificar los 10 parámetros desde un arranque ignorante con error máximo $\sim 1.1\%$. La Figura 3 muestra un ajuste sobre una trayectoria de test tipo *chirp* (frecuencia creciente), donde la identificación limpia reproduce fielmente la dinámica.

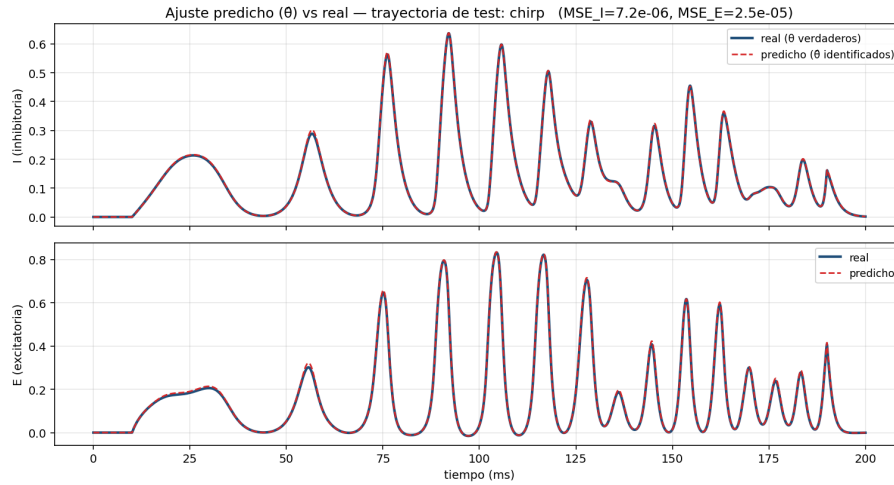


Figura 3: Ajuste del modelo identificado sobre una trayectoria de test tipo *chirp* (figura del proyecto): la dinámica aprendida generaliza a un estímulo de frecuencia variable no visto en entrenamiento.

La diferencia que decide el proyecto. La PINN mapea $t \mapsto \mathbf{x}$ para un estímulo *fijo*: no se le puede cambiar la entrada sobre la marcha, así que **no sirve como planta de un lazo de control**. La Neural ODE mapea $(\mathbf{x}, P, Q) \mapsto \dot{\mathbf{x}}$ con el estímulo como entrada explícita: *un solo modelo* responde a cualquier P, Q , por lo que puede integrarse en un lazo donde los estímulos se generan en línea. Por eso la Neural ODE es el **gemelo digital** de la planta y la que alimenta al controlador IMC (ver B5); la PINN es una excelente herramienta de *identificación* pero no de *control*.

Regla práctica. Usar **PINN** cuando el objetivo es identificar θ a partir de una o pocas trayectorias con ruido y no hace falta simular estímulos nuevos (denoising nativo, sin solver). Usar **Neural ODE** cuando se necesita un modelo *ejecutable* que responda a estímulos arbitrarios — identificación orientada al control, generación de trayectorias, gemelo digital.

7. Conexión con la identificabilidad y el control

Ni la PINN ni la Neural ODE eliminan por sí solas las *degeneraciones* del problema (p.ej. que w_{II} viva en una dirección casi plana del paisaje de pérdida). De eso se ocupa el análisis de Fisher+SVD (ver M6): decide *qué* parámetros son recuperables, *cuáles* fijar o regularizar y *qué* estímulo inyectar (diseño óptimo de experimentos, OED). Estas redes son el *motor de ajuste*; el diagnóstico de identificabilidad es el *mapa*.

El objetivo final es cerrar el lazo: los parámetros identificados por la Neural ODE alimentan un controlador IMC en lazo cerrado (ver B5), y la acción integral del controlador vuelve el seguimiento robusto al error residual de identificación — de ahí que la fragilidad de w_{II} no se propague al control. La implementación de todo esto (backbone gray-box, rollout RK4, softplus, multiple shooting) vive en el código `neural_ode` (ver P4).

Referencias

- [1] R. T. Q. Chen, Y. Rubanova, J. Bettencourt y D. Duvenaud, “Neural Ordinary Differential Equations”, *NeurIPS*, 2018.
- [2] M. Raissi, P. Perdikaris y G. E. Karniadakis, “Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear PDEs”, *J. Comput. Phys.*, 378:686–707, 2019.
- [3] C. Rackauckas *et al.*, “Universal Differential Equations for Scientific Machine Learning”, preprint arXiv:2001.04385, 2020.
- [4] H. G. Bock y K. J. Plitt, “A multiple shooting algorithm for direct solution of optimal control problems”, *IFAC Proceedings*, 17(2):1603–1608, 1984.
- [5] M. Tancik *et al.*, “Fourier features let networks learn high frequency functions in low dimensional domains”, *NeurIPS*, 2020.
- [6] I. Goodfellow, Y. Bengio y A. Courville, *Deep Learning*, MIT Press, 2016.